

ASSESSMENT TOOL - 8

Functional Dependencies and Normalization

1. Online Shopping Platform Normalize to 3NF

Relation:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Functional Dependencies

Assuming one order can contain multiple products:

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName, Price}$
- $(\text{OrderID, ProdID}) \rightarrow \text{Qty}$

Therefore:

- $\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$ — transitive dependency
- $\text{ProdID} \rightarrow \text{ProdName, Price}$

Candidate Key

(OrderID, ProdID)

1NF

All attributes contain atomic values, so the relation is already in **1NF**.

2NF

Remove partial dependencies from the composite key:

Customer

Customer(CustID, CustName)

Product

Product(ProdID, ProdName, Price)

Order

Order(OrderID, CustID)

OrderItem

OrderItem(OrderID, ProdID, Qty)

3NF

The above decomposition removes the transitive dependency $\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$.

Final 3NF schema:

- Customer(CustID, CustName)
- Product(ProdID, ProdName, Price)
- Order(OrderID, CustID)
- OrderItem(OrderID, ProdID, Qty)

Primary Keys:

- Customer $\rightarrow$ CustID
- Product $\rightarrow$ ProdID
- Order $\rightarrow$ OrderID
- OrderItem $\rightarrow$ (OrderID, ProdID)

2. Hospital Management Normalize to BCNF

Relation:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Functional Dependencies

Assuming:

- Each patient has one doctor.
- Each doctor has one name and specialization.
- Each ward has one ward name.

FDs:

- $PID \rightarrow PName, DocID, WardNo$
- $DocID \rightarrow DocName, Specialization$
- $WardNo \rightarrow WardName$

Candidate Key

PID

Anomalies

Insertion anomaly:

Cannot store a doctor or ward unless a patient is assigned.

Update anomaly:

If a doctor's specialization changes, several patient records may need updating.

Deletion anomaly:

Deleting the last patient assigned to a doctor may also remove the doctor's information.

BCNF Decomposition

Patient

Patient(PID, PName, DocID, WardNo)

Doctor

Doctor(DocID, DocName, Specialization)

Ward

Ward(WardNo, WardName)

In each relation, the determinant is a candidate key.

Final BCNF schema:

Patient(PID, PName, DocID, WardNo)

Doctor(DocID, DocName, Specialization)

Ward(WardNo, WardName)

3. University Enrollment 2NF Decomposition

Relation:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Functional Dependencies

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $(SID, CourseID) \rightarrow Grade$

Candidate Key

(SID, CourseID)

1NF

All values are atomic, so the relation is in **1NF**.

Partial Dependencies

Because the key is (SID, CourseID):

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$

These are partial dependencies.

2NF Decomposition

Student

Student(SID, SName)

Course

Course(CourseID, CourseName, Faculty)

Enrollment

Enrollment(SID, CourseID, Grade)

Now all non-key attributes depend on the whole key of their respective relation.

Final 2NF schema:

- Student(SID, SName)
- Course(CourseID, CourseName, Faculty)
- Enrollment(SID, CourseID, Grade)

4. Airline Booking Normalize up to BCNF

Relation:

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assume:

- A passenger ID uniquely identifies a passenger.
- A flight number determines its route.
- A flight on a particular date has a specific seat assignment for each passenger.

Functional Dependencies

- $\text{PassID} \rightarrow \text{PassName}$
- $\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$
- $(\text{FlightNo}, \text{Date}, \text{PassID}) \rightarrow \text{Seat}$

Candidate Key

(FlightNo, Date, PassID)

Decomposition

Passenger

Passenger(PassID, PassName)

Flight

Flight(FlightNo, DeptCity, ArrCity)

Booking

Booking(FlightNo, Date, PassID, Seat)

BCNF

- In Passenger, PassID is the key.
- In Flight, FlightNo is the key.
- In Booking, (FlightNo, Date, PassID) is the key.

Therefore the relations satisfy **BCNF** under the stated assumptions.

5. HR System Normalize to 3NF

Since the question does not provide a fixed relation, consider the following HR schema:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, LocationID, LocationName)

Functional Dependencies

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{ManagerID}, \text{LocationID}$
- $\text{DeptID} \rightarrow \text{DeptName}$
- $\text{ManagerID} \rightarrow \text{ManagerName}$
- $\text{LocationID} \rightarrow \text{LocationName}$

Transitive Dependencies

Examples:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

$\text{EmpID} \rightarrow \text{LocationID} \rightarrow \text{LocationName}$

Candidate Key

EmpID

3NF Decomposition

Employee

Employee(EmpID, EmpName, DeptID, ManagerID, LocationID)

Department

Department(DeptID, DeptName)

Manager

Manager(ManagerID, ManagerName)

Location

Location(LocationID, LocationName)

Result

The decomposition eliminates transitive dependencies and reduces redundancy.

6. Library System 4NF Decomposition

Relation:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Assume a member can independently have multiple books and multiple authors associated with their library records.

Functional Dependencies

- MemberID $\rightarrow$ MemberName
- BookID $\rightarrow$ Title, Genre

Suppose there are independent multivalued facts:

- MemberID $\twoheadrightarrow$ BookID
- BookID $\twoheadrightarrow$ Author

The MVDs can cause unnecessary combinations of values.

4NF Decomposition

Member

Member(MemberID, MemberName)

Book

Book(BookID, Title, Genre)

BookAuthor

BookAuthor(BookID, Author)

Borrowing

Borrowing(MemberID, BookID, BorrowDate)

Result

The independent multivalued relationships are separated, eliminating the redundancy caused by MVDs.

The resulting relations satisfy **4NF** under the stated assumptions.

7. Telecom Billing Candidate Keys and FDs

Consider the schema:

Billing(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Rate, Amount)

Functional Dependencies

- CustomerID $\rightarrow$ CustomerName
- CustomerID $\rightarrow$ PhoneNo
- PlanID $\rightarrow$ PlanName, Rate
- CallID $\rightarrow$ CustomerID, CallDate, Duration
- (CallID, PlanID) $\rightarrow$ Amount

If CallID uniquely identifies a call, then:

Candidate Key

CallID

Primary Key

CallID

Other Determinants

- CustomerID
- PlanID

Dependency Structure

CallID $\rightarrow$ CustomerID $\rightarrow$ CustomerName

CallID $\rightarrow$ PlanID $\rightarrow$ PlanName, Rate

These indicate transitive dependencies that should be removed during normalization.

Possible Normalized Design

Customer

Customer(CustomerID, CustomerName, PhoneNo)

Plan

Plan(PlanID, PlanName, Rate)

Call

Call(CallID, CustomerID, PlanID, CallDate, Duration, Amount)

8. Retail Inventory Complete Normalization

Relation:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Assume each product has one supplier and each supplier has one name.

Functional Dependencies

- $PID \rightarrow PName, SupplierID, Category, Price, Warehouse$
- $SupplierID \rightarrow SupplierName$

Therefore:

$PID \rightarrow SupplierID \rightarrow SupplierName$

This is a transitive dependency.

Candidate Key

PID

1NF

All attributes are atomic.

2NF

Since the key is a single attribute (PID), there are no partial dependencies.

Therefore, it is already in **2NF**.

3NF

Remove the transitive dependency.

Product

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Supplier

Supplier(SupplierID, SupplierName)

BCNF

Under the stated assumptions:

- PID determines all Product attributes.
- SupplierID determines SupplierName.

Both determinants are keys in their respective relations.

Thus the design satisfies **BCNF**.

9. School ERP — Normalize up to BCNF

Consider a poorly normalized relation:

School(SID, SName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks)

Functional Dependencies

- $SID \rightarrow SName, ClassID$
- $ClassID \rightarrow ClassName, TeacherID$
- $TeacherID \rightarrow TeacherName$
- $SubjectID \rightarrow SubjectName$
- $(SID, SubjectID) \rightarrow Marks$

Candidate Key

(SID, SubjectID)

Transitive Dependencies

$SID \rightarrow ClassID \rightarrow ClassName$

$SID \rightarrow ClassID \rightarrow TeacherID \rightarrow TeacherName$

$SubjectID \rightarrow SubjectName$

BCNF Decomposition

Student

Student(SID, SName, ClassID)

Class

Class(ClassID, ClassName, TeacherID)

Teacher

Teacher(TeacherID, TeacherName)

Subject

Subject(SubjectID, SubjectName)

Result

Marks(SID, SubjectID, Marks)

Final Design

Relation	Primary Key
Student	SID
Class	ClassID
Teacher	TeacherID
Subject	SubjectID
Marks	(SID, SubjectID)

Justification

The decomposition:

- Removes repeating and redundant information.
- Eliminates partial dependencies.
- Eliminates transitive dependencies.
- Preserves key constraints.
- Reduces insertion, update, and deletion anomalies.
- Produces a BCNF design under the stated assumptions.

10. Movie Streaming Platform 5NF

Consider a streaming platform relation:

Streaming(MovieID, ActorID, DirectorID, PlatformID)

Assume a movie can independently be associated with:

- multiple actors,
- multiple directors,
- multiple streaming platforms.

Multivalued Relationships

The relation may contain independent combinations such as:

MovieID $\rightarrow\rightarrow$ ActorID

MovieID $\rightarrow\rightarrow$ DirectorID

MovieID $\rightarrow\rightarrow$ PlatformID

However, if these associations are independent, storing them in one relation creates redundant combinations.

Join Dependency

The original relation can be represented as:

Streaming = MovieActor $\bowtie$ MovieDirector $\bowtie$ MoviePlatform

5NF Decomposition

MovieActor

MovieActor(MovieID, ActorID)

MovieDirector

MovieDirector(MovieID, DirectorID)

MoviePlatform

MoviePlatform(MovieID, PlatformID)

Lossless Join

The original relation can be reconstructed by joining the decomposed relations:

MovieActor $\bowtie$ MovieDirector $\bowtie$ MoviePlatform

If the original tuples represent exactly the valid combinations implied by these three relationships, the decomposition is **lossless**.

Final 5NF Design

- MovieActor(MovieID, ActorID)
- MovieDirector(MovieID, DirectorID)
- MoviePlatform(MovieID, PlatformID)

This removes redundancy caused by independent relationships and decomposes the relation into its irreducible join components.

Normalization Summary

Q	Case	Final Normal Form
1	Online Shopping	3NF
2	Hospital	BCNF
3	University Enrollment	2NF
4	Airline Booking	BCNF
5	HR System	3NF
6	Library	4NF
7	Telecom Billing	Candidate Keys + FDs
8	Retail Inventory	BCNF / Complete Normalization
9	School ERP	BCNF
10	Movie Streaming	5NF